

A Supreme Court Time Machine

```
In [1]: # Read SCOTUS database

import pandas as pd

cases_df = pd.read_csv('./data/SCDB_2023_01_caseCentered_Docket.csv', encoding='utf-8')
cases_df
```

Out[1]:

	caseId	docketId	caseIssuesId	voteld	dateDecision	decisionType	usCite	sc
0	1946-001	1946-001-01	1946-001-01-01	1946-001-01-01-01	11/18/1946	1	329 U.S. 1	
1	1946-002	1946-002-01	1946-002-01-01	1946-002-01-01-01	11/18/1946	1	329 U.S. 14	
2	1946-002	1946-002-02	1946-002-02-01	1946-002-02-01-01	11/18/1946	1	329 U.S. 14	
3	1946-002	1946-002-03	1946-002-03-01	1946-002-03-01-01	11/18/1946	1	329 U.S. 14	
4	1946-002	1946-002-04	1946-002-04-01	1946-002-04-01-01	11/18/1946	1	329 U.S. 14	
...	...	...	...	...	...	...	...	
10709	2022-058	2022-058-01	2022-058-01-01	2022-058-01-01-01	6/22/2023	1	599 U.S. 533	1
10710	2022-058	2022-058-02	2022-058-02-01	2022-058-02-01-01	6/22/2023	1	599 U.S. 533	1
10711	2022-059	2022-059-01	2022-059-01-01	2022-059-01-01-01	5/25/2023	1	598 U.S. 631	1
10712	2022-061	2022-061-01	2022-061-01-01	2022-061-01-01-01	5/22/2023	2	598 U.S. 623	1
10713	2022-062	2022-062-01	2022-062-01-01	2022-062-01-01-01	6/23/2023	1	599 U.S. 670	1

10714 rows x 53 columns

In [2]: *# Filter SCOTUS Database for relevant/useful columns*

```
cases_selected_columns_df = cases_df[['caseId', 'docketId', 'docket', 'term']  
cases_selected_columns_df.head()
```

Out[2]:

	caseId	docketId	docket	term	caseName	issueArea	partyWinning
--	--------	----------	--------	------	----------	-----------	--------------

0	1946-001	1946-001-01	24	1946	HALLIBURTON OIL WELL CEMENTING CO. v. WALKER e...	8.0	1.0
1	1946-002	1946-002-01	12	1946	CLEVELAND v. UNITED STATES	1.0	0.0
2	1946-002	1946-002-02	17	1946	CLEVELAND v. UNITED STATES	1.0	0.0
3	1946-002	1946-002-03	14	1946	CLEVELAND v. UNITED STATES	1.0	0.0
4	1946-002	1946-002-04	19	1946	CLEVELAND v. UNITED STATES	1.0	0.0

In [3]: *# Fetching the facts for the SCOTUS data was done outside of this notebook i*
the "oyez_script/py" file.

```
SCOTUS_df_unfiltered = pd.read_csv('./data/SCOTUS_data_with_facts')  
  
SCOTUS_df = SCOTUS_df_unfiltered[SCOTUS_df_unfiltered['facts'] != 'Facts of  
  
SCOTUS_df.loc[:, 'partyWinning'] = SCOTUS_df['partyWinning'].astype(bool)  
  
SCOTUS_df
```

Out[3]:

	Unnamed: 0	caseId	docketId	docket	term	caseName	issueArea	party
44	1308	1955-049	1955-049-01	10	1955	PENNSYLVANIA v. NELSON	3.0	
50	1315	1955-055	1955-055-01	23	1955	SLOCHOWER v. BOARD OF HIGHER EDUCATION OF NEW ...	3.0	
60	1326	1955-066	1955-066-01	95	1955	GRIFFIN et al. v. ILLINOIS	2.0	
91	1370	1955-097	1955-097-01	701	1955	REID, SUPERINTENDENT, DISTRICT OF COLUMBIA JAI...	2.0	
96	1376	1955-102	1955-102-01	442	1955	COLE v. YOUNG et al.	3.0	
...	...	...	...	...	...	...	...	...
7782	10708	2022-057	2022-057-01	22-210	2022	DUPREE v. YOUNGER	9.0	
7783	10709	2022-058	2022-058-01	22-381	2022	YEGIAZARYAN v. SMAGIN	8.0	
7784	10711	2022-059	2022-059-01	22-166	2022	TYLER v. HENNEPIN COUNTY, MINNESOTA	4.0	
7785	10712	2022-061	2022-061-01	22-714	2022	CALCUTT v. FDIC	8.0	
7786	10713	2022-062	2022-062-01	22-58	2022	UNITED STATES v. TEXAS	9.0	

3300 rows x 9 columns

In [4]: *# Read Kaggle dataset*

```
kaggle_df = pd.read_csv('./data/justice.csv', delimiter=',', encoding = "utf-8")  
kaggle_df.dropna(inplace=True)  
kaggle_df
```

Out [4]:

	Unnamed: 0	ID	name	href	docket	term
1	1	50613	Stanley v. Illinois	https://api.oyez.org/cases/1971/70-5014	70-5014	1971
2	2	50623	Giglio v. United States	https://api.oyez.org/cases/1971/70-29	70-29	1971
3	3	50632	Reed v. Reed	https://api.oyez.org/cases/1971/70-4	70-4	1971
4	4	50643	Miller v. California	https://api.oyez.org/cases/1971/70-73	70-73	1971
5	5	50644	Kleindienst v. Mandel	https://api.oyez.org/cases/1971/71-16	71-16	1971
...	...	...	...	...	...	...
3297	3297	63322	Yellen v. Confederated Tribes of the Chehalis ...	https://api.oyez.org/cases/2020/20-543	20-543	2020
3298	3298	63324	United States v. Palomar-Santiago	https://api.oyez.org/cases/2020/20-437	20-437	2020
3299	3299	63323	Terry v. United States	https://api.oyez.org/cases/2020/20-5904	20-5904	2020
3300	3300	63331	United States v. Cooley	https://api.oyez.org/cases/2020/19-1414	19-1414	2020
3302	3302	63335	PennEast Pipeline Co. v. New Jersey	https://api.oyez.org/cases/2020/19-1039	19-1039	2020

3098 rows x 16 columns

```
In [5]: # Filter Kaggle dataset for relevant/useful columns
```

```
kaggle_selected_columns_df = kaggle_df[['docket', 'term', 'name', 'facts', 'first_party_winner', 'issue_area']]
kaggle_selected_columns_df.head()
print(kaggle_selected_columns_df.sort_values(by=['term']))
```

	docket	term	name
\			
2900	837	1940-1955	Betts v. Brady
2458	10	1955	Pennsylvania v. Nelson
2462	23	1955	Slochower v. Board of Higher Education of New ...
2463	442	1955	Cole v. Young
2464	701	1955	Reid v. Covert
...	...	...	...
3267	19-508	2020	AMG Capital Management, LLC v. Federal Trade C...
3268	19-511	2020	Facebook, Inc. v. Duguid
3269	19-968	2020	Uzuegbunam v. Preczewski
3272	19-416	2020	Nestlé USA, Inc. v. Doe I
3302	19-1039	2020	PennEast Pipeline Co. v. New Jersey

	facts	facts_len	\
2900	<p>Betts was indicted for robbery in Maryland....	342	
2458	<p>Nelson, a member of the Communist Party, wa...	260	
2462	<p>New York City municipal charter provision §...	642	
2463	<p>The Food and Drug Administration ("FDA") fi...	877	
2464	<p>Mrs. Covert killed her husband on an airbas...	429	
...	...	...	
3267	<p>Scott Tucker owned several companies that p...	1658	
3268	<p>Noah Duguid brought this lawsuit because Fa...	1315	
3269	<p>In July 2016, Chike Uzuegbunam, a student a...	1814	
3272	<p>The plaintiff/respondents in this case are ...	2783	
3302	<p>The Natural Gas Act (NGA), 15 U.S.C. §§ 717...	1059	

	first_party_winner	issue_area
2900	True	Criminal Procedure
2458	False	First Amendment
2462	True	First Amendment
2463	True	First Amendment
2464	True	Civil Rights
...	...	...
3267	True	Economic Activity
3268	True	Economic Activity
3269	True	Judicial Power
3272	True	Judicial Power
3302	True	Due Process

[3098 rows x 7 columns]

```
In [6]: # Merge SCOTUS and Kaggle database
```

```
import numpy as np
```

```
# Standardize the docket values to all exist in the format yy-d
```

```

# where (y) is a year digit and (d) is a docket number between 1 and 9999
SCOTUS_df.loc[:, 'docket_standardized_SCOTUS'] = np.where(
    SCOTUS_df['docket'].str.contains('-'),
    SCOTUS_df['docket'],
    SCOTUS_df['docketId'].str[2:4] + '-' + SCOTUS_df['docket']
)

kaggle_selected_columns_df.loc[:, 'docket_standardized_kaggle'] = np.where(
    kaggle_selected_columns_df['docket'].str.contains('-'),
    kaggle_selected_columns_df['docket'],
    kaggle_selected_columns_df['term'].str[2:4] + '-' + kaggle_selected_colu
)

# Compute the merge
merged_df = SCOTUS_df.merge(kaggle_selected_columns_df, left_on='docket_star

```

/var/folders/kp/3kl_5m454dg835w66r844r3w0000gn/T/ipykernel_97159/3557145916.
py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

SCOTUS_df.loc[:, 'docket_standardized_SCOTUS'] = np.where(
    SCOTUS_df['docket'].str.contains('-'),
    SCOTUS_df['docket'],
    SCOTUS_df['docketId'].str[2:4] + '-' + SCOTUS_df['docket']
)

kaggle_selected_columns_df.loc[:, 'docket_standardized_kaggle'] = np.where(
    kaggle_selected_columns_df['docket'].str.contains('-'),
    kaggle_selected_columns_df['docket'],
    kaggle_selected_columns_df['term'].str[2:4] + '-' + kaggle_selected_colu
)

# Compute the merge
merged_df = SCOTUS_df.merge(kaggle_selected_columns_df, left_on='docket_star

```

/var/folders/kp/3kl_5m454dg835w66r844r3w0000gn/T/ipykernel_97159/3557145916.
py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

kaggle_selected_columns_df.loc[:, 'docket_standardized_kaggle'] = np.where(
    kaggle_selected_columns_df['docket'].str.contains('-'),
    kaggle_selected_columns_df['docket'],
    kaggle_selected_columns_df['term'].str[2:4] + '-' + kaggle_selected_colu
)

# Compute the merge
merged_df = SCOTUS_df.merge(kaggle_selected_columns_df, left_on='docket_star

```

In [7]:

```

# Consolidate merged dataframes
merged_df = merged_df.replace(np.nan, None)

merged_df['facts'] = np.where(merged_df['facts_x'], merged_df['facts_x'], merged_df['facts_y'])
merged_df['docket'] = np.where(merged_df['docket_standardized_SCOTUS'], merged_df['docket_standardized_SCOTUS'], merged_df['docket'])
merged_df['term'] = np.where(merged_df['term_x'], merged_df['term_x'], merged_df['term_y']).astype(str)
merged_df['first_party_winner'] = np.where(merged_df['partyWinning'], merged_df['partyWinning'], merged_df['first_party_winner'])

# Choose necessary columns from merged dataframe
df = merged_df[['docket', 'term', 'facts', 'first_party_winner']].copy()
print(df['first_party_winner'].dtype)

# Drop any row which does not have an outcome
df.dropna(subset=['first_party_winner'], inplace=True)

print(df)

```

	docket	term	facts
0	55-10	1955.0	Nelson, a member of the Communist Party, was c...
1	55-23	1955.0	New York City municipal charter provision §903...
2	55-95	1955.0	Judson Griffin and James Crenshaw were indicte...
3	55-701	1955.0	Mrs. Covert killed her husband on an airbase i...
4	55-442	1955.0	The Food and Drug Administration ("FDA") fired...
...	...	...	...
3317	88-325	1988	<p>In 1983, the American Trucking Associations...
3318	A-767	1991	<p>In 1979, a California state court convicted...
3319	98-405	1998	<p>Section 5 of the Voting Rights Act on 1995 ...
3320	10-1491	2011	<p>The Shell Petroleum Development Company of ...
3321	40-837	1940-1955	<p>Betts was indicted for robbery in Maryland....

	first_party_winner
0	False
1	True
2	True
3	True
4	True
...	...
3317	False
3318	True
3319	False
3320	False
3321	True

[3322 rows x 4 columns]

```
In [8]: # Get year from the term value column
def standardize_term(term):
    term = term[:4]
    return int(term)

df.loc[:, 'term'] = df['term'].apply(standardize_term)
```

```
In [9]: # NLP!!!

import warnings

warnings.filterwarnings("ignore", module="urllib3")

import nltk
from nltk.corpus import stopwords
import regex as re
import spacy

# remove tags
df.loc[:, 'facts'] = df['facts'].apply(lambda s: re.sub(r'<[^>]+>', '', s))

# format text
df.loc[:, 'facts'] = df['facts'].apply(lambda s: re.sub(r'^\w\s', '', s).lc

# remove stopwords
# nltk.download('stopwords')
```

```

stopwords_english = set(stopwords.words('english'))
df.loc[:, 'facts'] = df['facts'].apply(lambda s: ' '.join([word for word in
# tokenizer
nlp = spacy.load('en_core_web_sm', disable=['ner', 'parser'])

# lemmatize
df.loc[:, 'facts'] = [' '.join([word.lemma_ for word in facts]) for facts in
print(df['facts'])

```

```

0      nelson member communist party convict violate ...
1      new york city municipal charter provision 903 ...
2      judson griffin james crenshaw indict armed rob...
3      mrs covert kill husband airbase england pursua...
4      food drug administration fda fire kendrick col...

...

3317    1983 american trucking associations inc ata ch...
3318    1979 california state court convict robert all...
3319    section 5 voting right act 1995 prohibit bossy...
3320    shell petroleum development company nigeria lt...
3321    betts indict robbery maryland unable afford co...
Name: facts, Length: 3322, dtype: object

```

```

In [10]: # Given a year, determines a named court-era
def determine_era(year):
    year = int(year)
    if not year or year < 1942:
        return 'unk'
    elif 1942 <= year < 1986:
        return 'pre-rehnquist'
    elif 1986 <= year:
        return 'post-rehnquist_inc'

df.loc[:, 'era'] = df['term'].apply(determine_era)

```

```

In [11]: # Change first party winner to categorical variable for model training
df.loc[:, 'first_party_winner'] = df['first_party_winner'].astype('category')

# Save final full data
df.to_csv('./data/fully_processed_data')

# Filter data for non-recent and recent cases
df_pre_2021 = df[df['term'] < 2021]
df_post_2021_inc = df[df['term'] >= 2021]

# Save date-separated data
df_pre_2021.to_csv('./data/fully_processed_data_pre_2021')
df_post_2021_inc.to_csv('./data/fully_processed_data_post_2021_inc')

```

```

In [12]: # check sizes of each era's data
era_sizes = df_pre_2021['era'].value_counts().sort_index()
print(era_sizes)

# split into era
df_by_era = df_pre_2021.groupby(['era']).agg(list).reset_index()

```

```
df_by_era
```

```
# Save split-by-era data  
df_by_era.to_csv('./data/data_by_era_pre_2021')
```

```
era  
post-rehnquist_inc    2359  
pre-rehnquist         843  
unk                   1  
Name: count, dtype: int64
```

Machine Learning

```
In [13]: from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.pipeline import Pipeline  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.model_selection import GridSearchCV  
  
# Given an era and column name, gets the series corresponding to that column  
def get_column_value_by_era(era, column):  
    return df.loc[df['era'] == era, column]  
  
# Given an era, returns the X_train, X_test, y_train, y_test for that era's  
def train_test_split_for_era(era):  
    X = get_column_value_by_era(era, 'facts')  
    y = get_column_value_by_era(era, 'first_party_winner')  
  
    return train_test_split(X, y, test_size=0.2, random_state=4120)  
  
# Given a court era, trains a model on data from that era and scores the model  
def fit_and_score_model_for_era(era):  
    X_train, X_test, y_train, y_test = train_test_split_for_era(era)  
  
    param_grid = [  
        {'classifier': [RandomForestClassifier()], 'classifier__n_estimators': [100, 200, 400, 800]},  
        {'classifier': [LogisticRegression(max_iter=1000)], 'classifier__penalty': ['l1', 'l2']},  
        {'classifier': [KNeighborsClassifier()], 'classifier__n_neighbors': [3, 5, 7, 9, 11, 13, 15]}  
    ]  
  
    pipe = Pipeline(steps=[('cv', CountVectorizer()),  
                           ('classifier', KNeighborsClassifier())])  
  
    grid_search = GridSearchCV(pipe, param_grid, cv=5)  
    grid_search.fit(X_train, y_train)  
  
    return grid_search.best_estimator_, grid_search.best_score_
```

```
In [14]: # Fit and score both models  
pre_rehnquist_model, pre_rehnquist_model_score = fit_and_score_model_for_era('pre-rehnquist')  
post_rehnquist_model, post_rehnquist_model_score = fit_and_score_model_for_era('post-rehnquist')  
  
# Show Model Scores
```

```
print('Pre-rehnquist-model score: ', pre_rehnquist_model_score)
print('Post-rehnquist-inclusive-model score: ', post_rehnquist_model_score)
```

Pre-rehnquist-model score: 0.6632283029297954

Post-rehnquist-inclusive-model score: 0.6659898735465486

```
In [15]: # Download models
         from joblib import dump

         dump(pre_rehnquist_model, './saved_models/pre_rehnquist_model')
         dump(post_rehnquist_model, './saved_models/post_rehnquist_inc_model')
```

Out[15]: ['./saved_models/post_rehnquist_inc_model']